BRD_Agent_V2 – Developer Manual

Oct 16, 2026 · @Nikhil

BRD_Agent_V2 is a Python package that the HeyPM API calls to process meetings, answer chat questions, confirm JIRA actions and generate BRDs; your API wraps `service.py` and the UI reads one Firestore document per meeting.

1. Scope

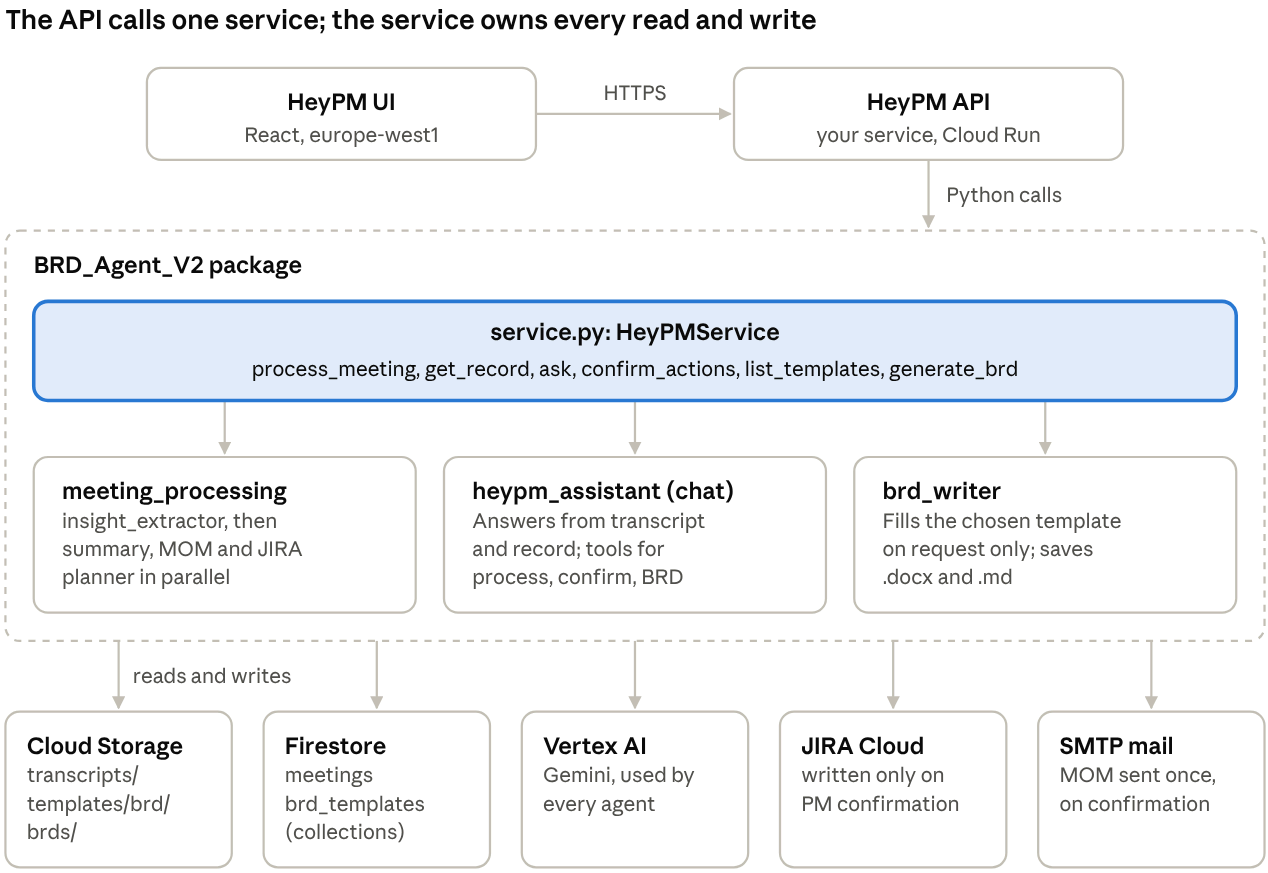
The agent owns the AI work and the writes to JIRA, email and Firestore; the API layer owns HTTP, auth, job handling and the UI.

The agent package does	The API / UI does
Read a meeting's transcript from GCS by meeting id	Decide when a meeting is processed (on open, or on upload)
Write summary, MOM and proposed JIRA actions to Firestore	Show the record, poll or listen for updates
Answer chat questions grounded in the transcript	Hold chat session ids per PM and meeting
Execute the stored JIRA plan and email the MOM on confirm	Show the plan, let the PM untick actions, call confirm
Fill a BRD template and save .docx/.md to GCS	Upload/manage templates, offer the BRD for download
Track timestamps and flags on the record	Authentication, authorisation, rate limits, CORS

Not in scope today: joining meetings or recording them ("Invite Notetaker"), project health scores, cross-meeting search. These sit on top of the same record and can be added later.

Code: `C:\Users\nikhi\Desktop\NIKHIL\GoogleAI\BRD_Agent_V2` (Python 3.11+, Google ADK 1.x, Gemini on Vertex AI, project `heypm-511616`).

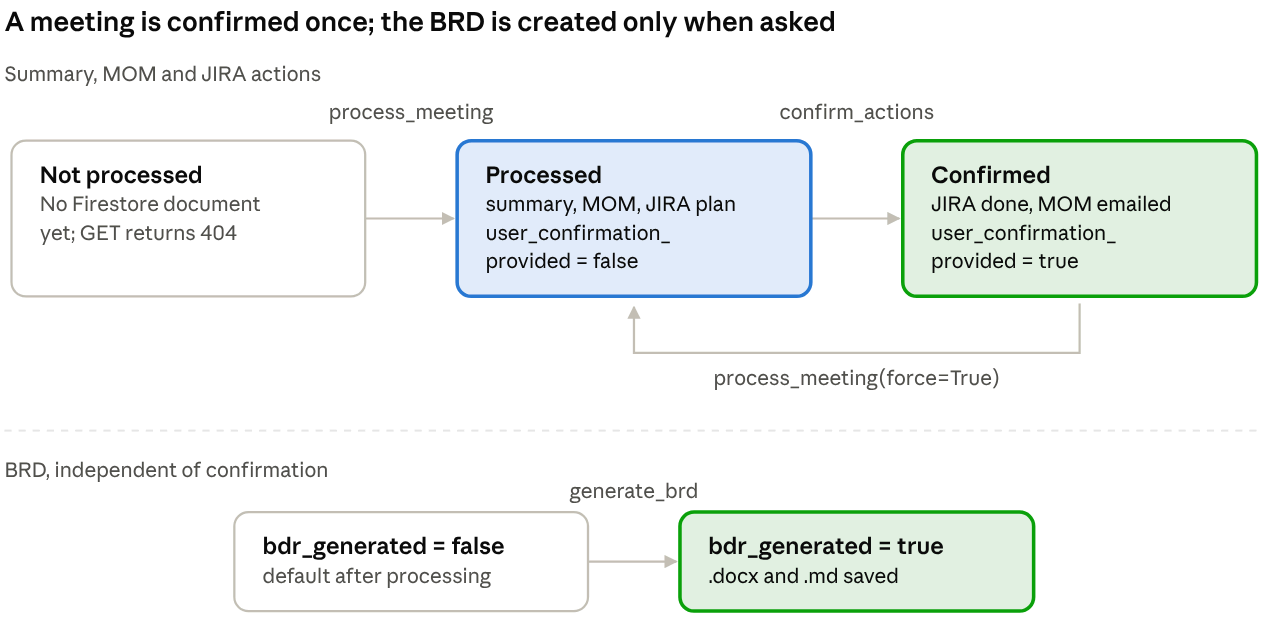
2. Architecture



architecture · UI, API, agent package, GCP services

Your API only talks to `service.py`; every read and write to GCS, Firestore, JIRA and email goes through it, and JIRA and email are touched only when the PM confirms.

3. Meeting record lifecycle



meeting record lifecycle · 3 states, BRD flag on its own

A record moves from processed to confirmed once; calling confirm again only retries failed or newly un-skipped actions, and regenerating after confirmation needs `force=True` and a fresh confirmation. The BRD flag is separate: it turns true whenever the PM asks for a BRD, before or after confirmation.

4. Firestore data contract

One document per meeting at `meetings/{meeting_id}` (collection name set by `FIRESTORE_MEETINGS_COLLECTION`). The UI reads only the top-level schema fields; the field names are kept exactly as agreed, including `jira_actions_requirend` and `bdr_generated`.

Schema fields (UI contract)

Field	Type	Written by	Meaning
meeting_id	string	process	Same as the document id and the transcript file name

Field	Type	Written by	Meaning
summary	string (Markdown)	process	One paragraph, then a "**Key points**" list of 4 to 7 bullets
MOM	string (Markdown)	process	Minutes email body: attendees, discussion, decisions, action-item table, open questions
jira_actions_requirend	boolean	process	`true` when the plan contains at least one action
jira_action_description	string (Markdown)	process	One bullet per planned action (A1, A2, …) with what, why and transcript timestamp; then "Not actioned" notes
bdr_generated	boolean	generate_brd	`false` until the PM asks for a BRD and it has been saved
relevant_jira	array of strings	process, confirm	Existing keys mentioned in the call; keys created on confirm are appended
user_confirmation_provided	boolean	confirm	`true` once the PM confirmed and JIRA ran without a blocking error
last_modified_date	string	every write	`YYYY-MM-DD` in APP_TIMEZONE (default Asia/Kolkata)
last_modified_time_stamp	string	every write	ISO 8601 with offset, e.g. `2026-10-16T09:51:02+05:30`

agent_data (internal, one nested map)

Everything the agent needs to resume work sits under `agent_data`, so the schema stays clean. The UI may read it for detail views but should not write to it.

Key	Meaning
status	`processed` or `confirmed`
transcript_uri	Where the transcript was read from (gs://… or a local path)
insights	Structured extraction: attendees, requirements, decisions, action items, open questions, JIRA requests
jira_plan	The typed plan executed on confirm: `project_key`, `actions[]` (ref, action, summary, parent, assignee, link_type, …), `notes[]`
jira_results	Per ref: `status` (ok, skipped, error), `action`, `key` for created issues, `error` / `reason`
jira_status / jira_message	Overall result: `ok`, `partial`, `dry_run`, `error`, `skipped`, with a message on error
skipped_refs	Refs the PM unticked on the last confirm
mom_email	`status` (sent, dry_run, error), `to`, `cc`, `subject`, `unresolved` names
brd	`docx_uri`, `md_uri`, `template_id`, `template_name`, `generated_at`, `warnings`
draft_warnings	Notes when a model output had to be cleaned or looked incomplete
processed_at / confirmed_at	ISO timestamps

Example document after process


```
{
  "meeting_id": "MTG-2026-10-15-INVOICE-ALERTS",
  "summary": "The team scoped SLA-based reminders ...\n\n**Key points**\n-
Teams first, email fallback\n- ...",
  "MOM": "Hi all,\n\nThanks for joining ...",
  "jira_actions_requirend": true,
  "jira_action_description": "Proposed JIRA actions for project FINAUTO:\n\n-
**A1** Create Epic: \"Invoice approval SLA alerts\" (13:52): asked by Rohan
...",
  "bdr_generated": false,
  "relevant_jira": ["FINAUTO-1098", "FINAUTO-1142"],
  "user_confirmation_provided": false,
  "last_modified_date": "2026-10-16",
  "last_modified_time_stamp": "2026-10-16T10:03:33+05:30",
  "agent_data": {"status": "processed", "jira_plan": {"project_key":
"FINAUTO", "actions": ["..."]}, "...": "..."}
}
```


5. Service API reference

All calls go through one object. Create it once per process; it builds the GCS, Firestore and template clients from the environment.

```
from BRD_Agent_V2.service import get_service, AlreadyConfirmed,
MeetingNotProcessed
from BRD_Agent_V2.stores import TranscriptNotFound, TemplateNotFound

svc = get_service()
```

The package folder's parent must be on `sys.path` (the package is imported as `BRD_Agent_V2`).

Method	Async	Typical time	Returns
process_meeting(meeting_id, force=False)	yes	30–90 s (3 model calls in parallel after 1 extraction)	Public record (schema fields only)

Method	Async	Typical time	Returns
get_record(meeting_id, include_agent_data=False)	no	<100 ms	Public record, or full record, or `None`
get_transcript(meeting_id)	no	<1 s	Normalised transcript text
ask(meeting_id, question, user_id="pm", session_id=None)	yes	3–10 s	`{session_id, answer, pending_confirmation}`
answer_confirmation(session_id, confirmation_id, confirmed, payload=None, user_id="pm")	yes	2–20 s	Same shape as `ask`
confirm_actions(meeting_id, skip_refs=None, send_email=True, email_overrides=None)	yes	2–20 s (JIRA + SMTP)	Full record
list_templates()	no	<1 s	`[{id, name, source, is_default}]`, bundled default last
generate_brd(meeting_id, template_id=None)	yes	40–120 s	Full record (`agent_data.brd` has the file URIs)

process_meeting

Reads the transcript, runs the extraction and drafting agents, and writes the record. Sets `user_confirmation_provided=false`. Keeps `bdr_generated` and any previously created JIRA keys.

- Raises `TranscriptNotFound` if no transcript exists for the id.
- Raises `AlreadyConfirmed` if the meeting was already confirmed; pass `force=True` to regenerate (the PM will need to confirm again; completed JIRA actions are not repeated).

confirm_actions

Call when the PM clicks Confirm. Executes the stored `jira_plan` and emails the MOM to attendees plus anyone the call asked to copy.

- `skip_refs`: refs the PM unticked, e.g. `["A3", "A4"]`. Actions that depend on a skipped one (a story under a skipped epic) are skipped too.
- `send_email=False` skips the MOM email.
- `email_overrides`: optional `{"subject", "to", "cc"}` to replace the computed values.
- Idempotent: actions already `ok` are not repeated, the MOM is sent once. Calling again retries failed or previously skipped actions.
- If an existing JIRA key in the plan cannot be found, nothing is changed: `agent_data.jira_status = "error"` and `user_confirmation_provided` stays false.
- Raises `MeetingNotProcessed` if the meeting has no plan yet.

generate_brd

Call only when the PM asks for a BRD. Processes the meeting first if needed, fills the template, saves `.docx` and `.md`, then sets `bdr_generated=true`.

- `template_id=None`: the PM's default template if one is marked, else the bundled default. `"default"`: always the bundled default. Otherwise an id or name from `list_templates()`.
- Raises `TemplateNotFound` for an unknown template.
- Check `agent_data.brd.warnings`: a non-empty list means the output was cleaned or looks incomplete.

ask / answer_confirmation

One chat turn. Pass the returned `session_id` back to continue a conversation. When the PM asks the chat to create JIRA items, `pending_confirmation` is `{confirmation_id, hint, payload}`; show `hint` (the plan) and call `answer_confirmation` with the PM's choice. Details in section 8.

6. Suggested REST endpoints

These are suggestions; the API shape is yours. Long-running calls (process, BRD) should run in the background and return 202, with the UI polling the record or listening to the Firestore document.

Method & path	Calls	Response
POST /meetings/{id}/process	process_meeting (background)	202; then GET the record
GET /meetings/{id}	get_record	Schema fields, 404 if not processed
GET /meetings/{id}/transcript	get_transcript	Plain text (Transcript tab)
POST /meetings/{id}/chat	ask	`{session_id, answer, pending_confirmation}`
POST /meetings/{id}/chat/confirm	answer_confirmation	Same as chat
POST /meetings/{id}/confirm	confirm_actions	Record with results
GET /templates	list_templates	Template list
POST /meetings/{id}/brd	generate_brd (background)	202; then GET the record and read `agent_data.brd`

Minimal FastAPI example

```
import sys
from pathlib import Path
from fastapi import BackgroundTasks, FastAPI, HTTPException
from pydantic import BaseModel

sys.path.insert(0, str(Path(__file__).resolve().parent))  # folder that
contains BRD_Agent_V2
from BRD_Agent_V2.service import AlreadyConfirmed, MeetingNotProcessed,
get_service
from BRD_Agent_V2.stores import TemplateNotFound, TranscriptNotFound

api = FastAPI()
svc = get_service()

class ChatIn(BaseModel):
    question: str
    session_id: str | None = None
```

```
class ConfirmIn(BaseModel):
    skip_refs: list[str] = []
    send_email: bool = True

class ChatConfirmIn(BaseModel):
    session_id: str
    confirmation_id: str
    confirmed: bool
    skip_refs: list[str] = []

class BrdIn(BaseModel):
    template_id: str | None = None

@api.post("/meetings/{mid}/process", status_code=202)
async def process(mid: str, bg: BackgroundTasks, force: bool = False):
    try:
        svc.get_transcript(mid)            # fail fast if the transcript is
missing
    except TranscriptNotFound as e:
        raise HTTPException(404, str(e))
    bg.add_task(svc.process_meeting, mid, force)
    return {"status": "processing"}

@api.get("/meetings/{mid}")
def record(mid: str):
    rec = svc.get_record(mid)
    if rec is None:
        raise HTTPException(404, "not processed")
    return rec

@api.post("/meetings/{mid}/chat")
async def chat(mid: str, body: ChatIn):
    return await svc.ask(mid, body.question, session_id=body.session_id)

@api.post("/meetings/{mid}/chat/confirm")
async def chat_confirm(mid: str, body: ChatConfirmIn):
    return await svc.answer_confirmation(body.session_id,
body.confirmation_id, body.confirmed,
                                         {"skip_refs": body.skip_refs})

@api.post("/meetings/{mid}/confirm")
async def confirm(mid: str, body: ConfirmIn):
```

```
    try:
        return await svc.confirm_actions(mid, skip_refs=body.skip_refs,
send_email=body.send_email)
    except MeetingNotProcessed as e:
        raise HTTPException(409, str(e))

@api.get("/templates")
def templates():
    return svc.list_templates()

@api.post("/meetings/{mid}/brd", status_code=202)
async def brd(mid: str, body: BrdIn, bg: BackgroundTasks):
    try:
        svc.stores.templates.get(body.template_id)   # validate before
starting
    except TemplateNotFound as e:
        raise HTTPException(404, str(e))
    bg.add_task(svc.generate_brd, mid, body.template_id)
    return {"status": "generating"}
```

Map `AlreadyConfirmed` to 409 if you expose `force`. For background work that must survive an instance restart, use Cloud Tasks or Pub/Sub instead of `BackgroundTasks`.

7. UI mapping

How each part of the HeyPM screens connects to the agent.

Screen element	Reads	Calls
Meetings list, click a meeting	`GET /meetings/{id}`; if 404, show "processing"	`POST /meetings/{id}/process` when no record exists
Meeting panel > Summary tab	`summary` (Markdown)	—

Screen element	Reads	Calls
Meeting panel > Transcript tab	transcript text	`GET /meetings/{id}/transcript`
Meeting panel > Actions tab	`jira_actions_requirend`, `jira_action_description`, `relevant_jira`, `user_confirmation_provided`; per-action checkboxes from `agent_data.jira_plan.actions[].ref`	Confirm button: `POST /meetings/{id}/confirm` with unticked refs as `skip_refs`
MOM view	`MOM` (Markdown)	Sent on confirm; status in `agent_data.mom_email`
"Action items taken" card in chat	`relevant_jira`, `agent_data.jira_results` (keys and statuses)	—
Chat box	answers	`POST /meetings/{id}/chat`, keeping `session_id` per PM and meeting
Chat confirm card	`pending_confirmation.hint`	`POST /meetings/{id}/chat/confirm`
Templates tab	template list	`GET /templates`; upload writes to the template store (section 9)
"Generate BRD" / "View BRD"	`bdr_generated`, `agent_data.brd.docx_uri`	`POST /meetings/{id}/brd`; serve the file through a signed URL

Show the Confirm button only while `jira_actions_requirend` is true and `user_confirmation_provided` is false; after confirm, show `agent_data.jira_results`, including any `error` rows, with a retry that calls confirm again.

8. Chat integration

The chat area is backed by an ADK agent, `heypm_assistant`. Each chat session is tied to one meeting.

What the chat agent sees on every turn: the full transcript (loaded once per session from GCS) and a fresh copy of the Firestore record (re-read each turn, so changes made through the UI are visible). It answers only from these two sources, cites speakers and timestamps, and says so when the answer isn't there.

What it can do when asked: open another meeting, (re)process the meeting, create the JIRA items and email the MOM (after confirmation), list templates, and generate a BRD. Each action goes through the same service methods as the API, so the Firestore record stays consistent whichever path the PM uses.

Sessions

- First message: call `ask(meeting_id, question)` without `session_id`. Store the returned `session_id` against the PM and meeting.
- Later messages: pass that `session_id` so the conversation keeps its history.
- Sessions are held in memory in `HeyPMService` by default. That works for one instance only; it is lost on restart and not shared across Cloud Run instances. For production, change `HeyPMService._chat()` to build the Runner with a persistent session service: `DatabaseSessionService(db_url=...)` on Cloud SQL, or `VertexAiSessionService` (Agent Engine).

Confirmation round-trip

When the PM types something like "create the JIRA stories", the agent pauses and asks for confirmation instead of acting:

1. `ask` returns `pending_confirmation = {confirmation_id, hint, payload}`. `hint` is the plan as text (one line per action with reason and timestamp); `payload` is `{"skip_refs": [...]}`.
2. Show the plan in a card with Confirm / Cancel and checkboxes for refs.
3. Call `answer_confirmation(session_id, confirmation_id, confirmed, {"skip_refs": [...]})`.
4. The agent runs `confirm_actions` and replies with the outcome; the Firestore record is updated as with the Confirm button.

If you'd rather keep all JIRA writes on the Confirm button, have the UI ignore chat confirmations and remove `confirm_jira_actions` from the agent's tool list in `agent.py`.

9. Transcripts and templates

Transcripts in GCS

The meeting id is the lookup key. For `meeting_id = MTG-123` the agent tries, in order:

1. `gs://TRANSCRIPT_BUCKET/TRANSCRIPT_PREFIX/MTG-123.txt`, then `.vtt`, `.json`, `.md`
2. The first transcript file inside the folder `gs://TRANSCRIPT_BUCKET/TRANSCRIPT_PREFIX/MTG-123/`
3. If the meeting id itself is a full `gs://` URI, that object

Whatever writes transcripts (the notetaker, an upload, a Teams/Meet export job) only has to follow this naming. Supported formats, all normalised to `Speaker m:ss` blocks before the model sees them:

Format	Shape	Typical source
.txt / .md	`Name m:ss` line, then what was said	Teams "Download transcript"
.vtt	WebVTT cues, speaker in `<v Name>` or `Name:`	Teams, Google Meet, Zoom exports
.json	List of `{speaker, start, text}`, or `{"segments" | "utterances": [...]}`; `start` in seconds or text	Speech-to-text output, custom notetakers

BRD templates

The storage choice is still open, so the agent supports both options at once (`TEMPLATE_SOURCE=auto`). Pick one when the Templates tab is built.

Option	Where	Shape	Default template
Firestore	Collection `brd_templates`	`{"name": "Finance BRD", "content": "<markdown>", "is_default": false}`, or `gcs_uri` pointing at a .docx/.md instead of `content`	`is_default: true`
GCS	`gs://TEMPLATE_BUCKET/templates/brd/`	`.docx`, `.md` or `.txt` files; file name = template name	File named `default.*`
Bundled	`templates/default_brd_template.md` in the package	20-section BRD	Used when no user default exists

Template ids returned by `list_templates()`: `firestore:<doc id>`, `gcs:<object path>`, `default`. `generate_brd` also accepts a template's display name.

How a template is used: Word templates are converted to a Markdown skeleton (Heading 1/2/3 become sections; guidance paragraphs and table header rows are kept). The BRD writer keeps the template's sections, order and table columns, replaces placeholders like `<...>` and guidance text with content from the meeting, writes "TBC" for anything not discussed (and lists it as an open question), and never invents JIRA keys. If the template's last section is missing from the output, `agent_data.brd.warnings` says so.

Guidance for template authors: use real heading styles, keep section names short, put instructions for the writer as a sentence under each heading, and give tables a header row only.

10. Configuration reference

All settings come from environment variables (locally from `BRD_Agent_V2/.env`; on Cloud Run from the service's env vars and Secret Manager).

Variable	Default	Purpose
GOOGLE_GENAI_USE_VERTEXAI	—	`TRUE` to call Gemini through Vertex AI
GOOGLE_CLOUD_PROJECT	—	`heypm-511616`
GOOGLE_CLOUD_LOCATION	—	Vertex region for the model, e.g. `us-central1` or `global`
GEMINI_MODEL	gemini-2.5-flash	Model for every agent
STORAGE_BACKEND	local	`gcp` = GCS + Firestore; `local` = folders under `local_data/`
TRANSCRIPT_BUCKET	—	Bucket holding transcripts (required for gcp)
TRANSCRIPT_PREFIX	transcripts	Folder inside the bucket
FIRESTORE_DATABASE	(default)	Firestore database id
FIRESTORE_MEETINGS_COLLECTION	meetings	Collection for meeting records
FIRESTORE_TEMPLATES_COLLECTION	brd_templates	Collection for templates (Firestore option)
TEMPLATE_SOURCE	auto	`auto`, `firestore`, `gcs` or `default`
TEMPLATE_BUCKET	TRANSCRIPT_BUCKET	Bucket for template files
TEMPLATE_PREFIX	templates/brd	Folder for template files
OUTPUT_BUCKET	TRANSCRIPT_BUCKET	Bucket for generated BRDs
OUTPUT_PREFIX	brds	BRDs saved at `<prefix>/<meeting_id>/BRD-<title>-<yyyymmdd-hhmm>.docx` and `.md`
APP_TIMEZONE	Asia/Kolkata	Time zone for `last_modified_*`
PEOPLE_FILE	people.yaml	Name to email / JIRA account mapping

Variable	Default	Purpose
DRY_RUN	true	`true`: confirm runs fully but sends no email and doesn't touch JIRA
JIRA_BASE_URL, JIRA_EMAIL, JIRA_API_TOKEN	—	JIRA Cloud credentials (token in Secret Manager)
JIRA_PROJECT_KEY	FINAUTO	Project for new issues and bare ticket numbers said in the call
SMTP_HOST, SMTP_PORT, SMTP_STARTTLS	—, 587, true	Mail server
SMTP_USERNAME, SMTP_PASSWORD, MAIL_FROM	—	Mail account (password in Secret Manager)

people.yaml maps names spoken in meetings to identities. The agent never guesses an email or JIRA account:

```
people:
  - name: Karthik Iyer
    email: karthik.iyer@company.com
    jira_account_id:        # optional; looked up by email if blank
    aliases: [Karthik]
```

In production this can move to Firestore or the company directory; only `directory.py` needs to change.

11. Deploying on Cloud Run

The package has no web server of its own; it is imported by your API service. Deploy the API with the package folder next to it.

Service account roles (on project heypm-511616 unless noted):

Role	Needed for
Vertex AI User	Calling Gemini
Cloud Datastore User	Reading and writing Firestore
Storage Object Viewer on the transcript bucket	Reading transcripts and templates
Storage Object Creator on the output bucket	Writing BRDs

Role	Needed for
Secret Manager Secret Accessor	JIRA token and SMTP password

Settings to get right:

- Request timeout: at least 300 s if process or BRD run inside the request; better, run them as background jobs (Cloud Tasks or Pub/Sub) and keep requests short.
- Memory: 1 GiB is enough; CPU always allocated if you use in-process background tasks.
- Secrets: map `JIRA_API_TOKEN` and `SMTP_PASSWORD` from Secret Manager to env vars; never bake them into the image.
- `STORAGE_BACKEND=gcp`, bucket names, and `DRY_RUN=false` only once JIRA and SMTP are verified.
- Chat sessions: use a persistent session service when running more than one instance (section 8).
- CORS: allow the HeyPM UI origin.
- Region: the UI runs in europe-west1; Firestore and the buckets can sit in Europe while the model is called in `us-central1` or `global`.

Downloads: BRDs are private objects in GCS (`gs://…`). Give the UI a short-lived signed URL rather than making the bucket public.

Example deploy (from the API folder that contains BRD_Agent_V2):

```
gcloud run deploy heypm-api --source . --region europe-west1 --project heypm-
511616 \
  --service-account heypm-api@heypm-511616.iam.gserviceaccount.com \
  --set-env-vars GOOGLE_GENAI_USE_VERTEXAI=TRUE,GOOGLE_CLOUD_PROJECT=heypm-
511616,GOOGLE_CLOUD_LOCATION=us-
central1,STORAGE_BACKEND=gcp,TRANSCRIPT_BUCKET=<bucket>,DRY_RUN=true \
  --set-secrets JIRA_API_TOKEN=jira-token:latest,SMTP_PASSWORD=smtp-
password:latest \
  --timeout 600 --memory 1Gi
```

12. Guarantees, errors and troubleshooting

What the agent guarantees

- **Grounded output.** Summary, MOM, plan, BRD and chat answers use only the transcript; unresolved points become open questions, never guesses.
- **No guessed identities.** Email recipients and JIRA assignees come only from people.yaml; unknown names are reported in `mom_email.unresolved` or a `note` on the JIRA result.
- **The plan that was shown is the plan that runs.** The model drafts a typed plan at process time; on confirm, code executes exactly that stored plan. The model never calls JIRA directly.
- **All or nothing on bad references.** Before any JIRA write, every existing key in the plan (minus skipped actions) must exist and be visible; otherwise nothing changes.
- **Idempotent confirm.** Completed actions are not repeated; the MOM is sent at most once.
- **Bounded output.** Drafts are capped in length and cleaned (runaway table padding is a known Gemini failure); problems are recorded in `draft_warnings` / `brd.warnings`.
- **Safe default.** `DRY_RUN=true` exercises the whole flow with placeholder keys (`FINAUTO-NEW1`) and no email.

Errors

Error / state	When	Suggested HTTP	What to do
TranscriptNotFound	No transcript for the id	404	Check bucket, prefix and file name
MeetingNotProcessed	Confirm before process	409	Call process first
AlreadyConfirmed	Process after confirm	409	Offer "regenerate" with `force=True`
TemplateNotFound	Unknown template id/name	404	Refresh the template list

Error / state	When	Suggested HTTP	What to do
`agent_data.jira_status = error`	Missing JIRA key, bad credentials	200 with status	Show `jira_message`; fix and confirm again
`jira_results[ref].status = error`	One action failed (permissions, workflow)	200 with status	Show the row; confirm again retries it
`mom_email.status = error`	SMTP failure or no resolvable recipients	200 with status	Fix SMTP or people.yaml
RuntimeError ("did not produce…")	Model returned nothing usable	502	Retry; check Vertex quota and logs
google.api_core exceptions	GCS / Firestore / Vertex permission or quota	502/503	Check service account roles

Troubleshooting

Symptom	Likely cause	Fix
403 from Vertex	Missing Vertex AI User role or API disabled	Grant role; enable aiplatform.googleapis.com
404 model not found	Model not in GOOGLE_CLOUD_LOCATION	Use `global` or another region
Processing takes >2 min	Very long transcript or slow region	Expected for 1-hour calls; run as a background job
BRD warning "looks incomplete"	Output hit the length cap	Regenerate; shorten the template if it repeats
Chat forgets earlier turns	New session_id each call, or instance restart	Reuse session_id; use a persistent session service

Symptom	Likely cause	Fix
JIRA link direction looks reversed	JIRA's inward/outward wording for the link type	Verify once on your instance; swap in `jira_client.link` if needed

13. Code layout, tests and extension points

Layout

Path	What it holds
service.py	`HeyPMService`: the API surface (process, ask, confirm, templates, BRD)
agent.py	Chat agent `heypm_assistant`, its tools, and the ADK `App` named `BRD_Agent_V2`
pipeline.py	Workflow agents: insight_extractor, summary_writer, mom_writer, jira_planner, brd_writer; output cleanup
prompts.py	All instructions (grounding and table rules shared)
schemas.py	Pydantic models: `MeetingInsights`, `JiraPlan`
stores/	transcripts.py (GCS/local + format parsing), meetings.py (Firestore/local), templates.py, files.py
tools/	jira_ops.py (plan description, execution, dry run), jira_client.py (JIRA Cloud REST v3), email_ops.py (SMTP), docx_render.py
templates/default_brd_template.md	Bundled default BRD template
people.yaml, directory.py	Name to email / JIRA account lookup
run.py	CLI: process, show, ask, chat, confirm, templates, brd
tests/test_v2.py	Offline tests

Tests

```
cd BRD_Agent_V2
pip install pytest pytest-asyncio
pytest -q
```

Eight offline tests, no GCP or model calls: a scripted model stands in for Gemini, and fakes stand in for GCS and Firestore. They cover the full process → confirm → BRD flow, the exact schema fields, idempotent confirm, chat Q&A, the chat confirmation round-trip, template selection from .md and .docx, and transcript parsing (.txt, .vtt, .json). Add a test here for any behaviour the UI relies on.

Extension points

Change	Where
Different model per step	`model=` on each agent in pipeline.py / agent.py
Outlook / Gmail API instead of SMTP	`_smtp_send` in tools/email_ops.py
People from Firestore or the directory	directory.py `load_people()`
Persistent chat sessions	`HeyPMService._chat()`
Publish the BRD to Confluence or Google Docs	After `files.save` in `generate_brd`
New output (e.g. Status Report template)	New agent in pipeline.py + service method, same pattern as `generate_brd`
Status while processing	Write `agent_data.status = "processing"` at the start of `process_meeting`

Open decisions

Decision	Current behaviour	Owner
Template storage	Both Firestore and GCS supported (`TEMPLATE_SOURCE=auto`)	Product / UI
Where transcripts come from	Any writer following the GCS naming in section 9	API / notetaker

Decision	Current behaviour	Owner
Chat-initiated JIRA writes	Allowed with confirmation	Product
BRD download	`gs://` URIs stored; signed URLs to be issued by the API	API
People directory	people.yaml in the package	Platform